第8课 while循环、break与continue

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：用 while 从 1 加到 100（入门级——和 for 对比）

讲解思路：

> 上节课我们用 for 做过"从 1 加到 100"，今天我们用 while 来做一遍，你会发现——它们其实可以互相改写！

完整代码：

```
#include <iostream>
using namespace std;

int main() {
    // --- for 写法（回顾）---
    // int sum = 0;
    // for (int i = 1; i <= 100; i++) {
    //     sum += i;
    // }

    // --- while 写法 ---
    int sum = 0;
    int i = 1;          // 第1步：初始化写在 while 前面
    while (i <= 100) {  // 第2步：条件写在括号里
        sum += i;
        i++;             // 第3步：更新写在循环体里（千万别忘！）
    }
    cout << "1到100的和是：" << sum << endl;
    return 0;
}
```

输出：

```
1到100的和是：5050
```

讲解要点：
- 引导学生对比 for 和 while 的三部分在哪里。
- 强调 `i++` 如果忘写就会死循环——让学生体验一下 Ctrl+C。

例题 2：不断输入，直到输入 0 停止（哨兵值——while 的经典用法）

讲解思路：

> 想象你是一个小商店的收银员。顾客一件一件买东西，你每次输入一个价格，最后输入 0 表示"结账了"，程序算出总价。
>
> 这个题你**不知道顾客要买多少件**，所以不能用 for——用 while 最合适！
>
> 这里的 0 就叫**"哨兵值"**——像哨兵一样守在门口，看到它就知道该结束了。

完整代码：

```
#include <iostream>
using namespace std;

int main() {
    int price;
    int total = 0;

    cout << "请输入商品价格（输入0表示结账）：" << endl;
    cin >> price;             // 先读一个

    while (price != 0) {     // 只要不是0，就继续
        total += price;       // 加到总价里
        cin >> price;         // 再读下一个
    }

    cout << "总价是：" << total << "元" << endl;
    return 0;
}
```

示例运行：

```
请输入商品价格（输入0表示结账）：
10
25
8
0
总价是：43元
```

讲解要点：
- 强调"先读一次，再进循环"的模式（避免 0 也被加进去）。
- 让学生口头模拟执行过程：输入 10 不等于 0 加到 total 输入 25 …… 输入 0 等于 0 循环结束。
- 引出"哨兵值"的概念：0 不是数据，而是一个停止信号。

例题 3：用 break 寻找第一个能同时被 3 和 7 整除的数（break 示例）

讲解思路：

> 题目：从 1 开始找，找到**第一个**能同时被 3 和 7 整除的数，输出它，然后停下来。
> 我们不知道这个数是多少，但知道一定能找到。用 while(true) 加上 break，就像是"**一直找，找到就从紧急出口跑出来**"。

完整代码：

```
#include <iostream>
using namespace std;

int main() {
    int n = 1;

    while (true) {            // "一直找"——这是一个有 break 守护的死循环
        if (n % 3 == 0 && n % 7 == 0) {
            cout << "找到了：" << n << endl;
            break;            // 找到了，紧急出口！
        }
        n++;
    }
    return 0;
}
```

输出：

```
找到了：21
```

讲解要点：
- `while (true)` 看起来是死循环，但有了 break 就不怕。
- 如果没有 break，程序会一直运行下去——让学生感受 break 的价值。
- 引导思考：这题也可以把条件写在 while 里，但用 break 写法更直观，尤其"找第一个满足条件的"场景。

例题 4：输出 1 到 20，但跳过所有 3 的倍数（continue 示例）

讲解思路：

> 想象你在报数：1、2、3、4、5……但老师说"**凡是 3 的倍数不许报！**"那遇到 3、6、9……就闭嘴跳过，继续往下报。
> 这就是 continue 的作用——"**这个数不算，跳过去，继续下一个**"。

完整代码：

```
#include <iostream>
using namespace std;

int main() {
    int i = 0;

    while (i < 20) {
        i++;                    // 注意：i++ 放在 continue 前面！
        if (i % 3 == 0) {
            continue;           // 跳过本轮，不执行下面的 cout
        }
        cout << i << " ";
    }
    cout << endl;
    return 0;
}
```

输出：

```
1 2 4 5 7 8 10 11 13 14 16 17 19 20
```

讲解要点：
- **重点强调**：`i++` 必须放在 continue 的**前面**！如果放后面，遇到 3 的倍数时 continue 跳过了 `i++`，i 永远不变 死循环！
- 这是 while + continue 最容易出错的地方，让学生亲手把 `i++` 放到 continue 后面试一试，感受死循环，加深印象。
- 和 break 对比：break 是"我走了不玩了"，continue 是"这轮跳过，我还继续玩"。

附：例题源代码与运行结果

例题 1：Sum 1 to 100 Using while (Compare with for)

```
// Compute 1 + 2 + 3 + ... + 100 using a while loop.
#include <iostream>
using namespace std;

int main() {
    int sum = 0;
    int i = 1;            // Step 1: Initialize before while
    while (i <= 100) {    // Step 2: Condition in parentheses
        sum += i;
        i++;              // Step 3: Update inside the loop body (don't forget!)
    }
    cout << "Sum from 1 to 100: " << sum << endl;
    return 0;
}
```

Expected Output

```
Sum from 1 to 100: 5050
```

例题 2：Keep Reading Until 0 (Sentinel Value)

```
// Simulate a cashier: read prices one by one, input 0 to checkout.
// Output the total price.
#include <iostream>
using namespace std;

int main() {
    int price;
    int total = 0;

    cout << "Enter prices (0 to checkout):" << endl;
    cin >> price;              // Read the first price

    while (price != 0) {      // Continue as long as it's not 0
        total += price;        // Add to total
        cin >> price;          // Read next price
    }

    cout << "Total: " << total << endl;
    return 0;
}
```

Sample Input

```
10
25
8
0
```

Expected Output

```
Enter prices (0 to checkout):
Total: 43
```

例题 3：Find First Number Divisible by Both 3 and 7 (break)

```
// Starting from 1, find the first number divisible by both 3 and 7.
#include <iostream>
using namespace std;

int main() {
    int n = 1;

    while (true) {                      // Keep searching
        if (n % 3 == 0 && n % 7 == 0) {
            cout << "Found: " << n << endl;
            break;                      // Exit immediately
        }
        n++;
    }
    return 0;
}
```

Expected Output

```
Found: 21
```

例题 4：Print 1 to 20 but Skip Multiples of 3 (continue)

```
// Output numbers from 1 to 20, skipping all multiples of 3.
#include <iostream>
using namespace std;

int main() {
    int i = 0;

    while (i < 20) {
        i++;                  // Update BEFORE continue to avoid infinite loop!
        if (i % 3 == 0) {
            continue;         // Skip this iteration, go back to condition check
        }
        cout << i << " ";
    }
    cout << endl;
    return 0;
}
```

Expected Output

```
1 2 4 5 7 8 10 11 13 14 16 17 19 20
```